

Research Note: Low-Level Design (LLD) Practice & Evaluation Platform

- **Author:** Amitesh Kumar Dubey
- **Date:** 23 September 2026
- **Status:** MVP Prototype — Assignment Submission

1. The Learner Problem in Low-Level Design (LLD)

Low-Level Design (LLD) — also referred to as Object-Oriented Design (OOD) — is a critical engineering competency required for constructing scalable, maintainable, and extensible software. In technical interviews and real-world software engineering, LLD evaluates how well an engineer can translate high-level product requirements into clean class hierarchies, precise interfaces, solid encapsulation, and appropriate design patterns (e.g., Strategy, Factory, Observer, State).

Despite its importance, software engineers face a fundamental bottleneck when attempting to practice LLD: **the practice loop is easy to start, but extremely hard to evaluate.**

When a learner tackles classic LLD problems — such as designing a *Parking Lot*, an *Elevator Control System*, a *Vending Machine*, or a *Distributed Rate Limiter* — they can draft UML class diagrams or write skeleton code. However, once the code is drafted, learners hit a wall:

1. **Uncertainty of Quality:** *"Is my `Vehicle` class hierarchy over-engineered?", "Did I break the Single Responsibility Principle (SRP) in `ParkingLot`?", "Should fee calculation be a strategy interface or a method on `Ticket`?"*
2. **Lack of Automated, Explainable Feedback:** Unlike Data Structures & Algorithms (DSA), where a solution either passes test cases or fails, LLD solutions cannot be validated with binary assertion suites alone. Multiple valid designs exist for any given LLD problem.
3. **Absence of Iterative Feedback Loops:** Current practice methods offer no actionable path for a learner to submit a design, receive structured structural and qualitative feedback, refine their abstractions, and resubmit to measure improvement over time.

2. Analysis of Existing Tools & Approaches

To understand where the current ecosystem fails LLD learners, we evaluated three existing approaches:

Approach / Tool Category	How It Works	Key Strengths	Fundamental Gaps & Flaws for LLD
1. DSA & Code Execution Platforms (LeetCode, HackerRank, CodeSignal)	Executes submitted code against unit test suites and input/output assertions.	Instant pass/fail results, precise algorithmic efficiency metrics ($O(N)$ runtime).	Completely unfit for LLD. These platforms evaluate <i>functional correctness</i> , not <i>design quality</i> . A learner can solve a Parking Lot in a single 500-line monolithic <code>main()</code> function with global arrays and pass unit tests while writing atrocious object-oriented design.
2. Static Analysis & Linters (SonarQube, ESLint, Checkstyle, PMD)	Scans codebases against predefined rule sets for code smells, syntax errors, and complexity metrics.	Detects syntax flaws, unused variables, and basic circular dependencies.	Lacks architectural context. Linters cannot evaluate intent. They cannot determine whether an abstraction (e.g., <code>ElevatorDispatcher</code>) appropriately delegates work, whether domain relationships are properly encapsulated, or if a design violates SOLID principles.

Approach / Tool Category	How It Works	Key Strengths	Fundamental Gaps & Flaws for LLD
3. Traditional Code Reviews / Peer Feedback (GitHub Pull Requests, Discord/Slack communities)	Human engineers review code/diagrams manually and provide textual feedback.	High qualitative value, nuance, real-world engineering perspective.	Non-scalable, slow, and inconsistent. Feedback takes hours or days. Reviewers vary widely in experience, often giving conflicting advice without a standardized rubric.

3. Key Gaps Identified

From our research, four key gaps prevent effective LLD learning:

- Gap 1: Absence of Hybrid Evaluation (Structural + Open-Ended Reasoning)**
Neither pure assertion testing nor generic text linters work in isolation. Effective feedback requires a **hybrid engine**: deterministic structural rules (checking class counts, interfaces, enums, SOLID anti-patterns) combined with LLM-driven reasoning for qualitative design assessment.
- Gap 2: Lack of Context-Aware Rubrics**
Generic code advice fails because different LLD problems demand different design patterns. A *Vending Machine* problem requires evaluating the State Pattern, whereas a *Parking Lot* requires evaluating Strategy for pricing and Factory for spot assignment. Rubrics must be domain-aware.
- Gap 3: Missing Iterative Attempt Tracking**
Learners improve by iterating on suggestions. Without tracking attempt histories and comparing design scores across iterations, learners cannot visualize their growth.
- Gap 4: Fragility in Automated Evaluation Systems**
Large Language Model (LLM) calls can be slow, rate-limited, or occasionally fail. Without fallback evaluation mechanisms and robust state handling (`IN_PROGRESS` $\rightarrow$ `EVALUATING` $\rightarrow$ `COMPLETED` / `EVALUATION_FAILED`), platforms crash or hang, frustrating users.

4. Product Direction & Core Principles

To bridge these gaps, we present the **LLD Practice Platform**, built around the core learner practice loop:

\$\$\text{Choose Problem} \rightarrow \text{Design Solution} \rightarrow \text{Submit} \rightarrow \text{Get Feedback} \rightarrow \text{Review} \rightarrow \text{Iterate \& Retry}\$\$

Product Guiding Principles:

- Domain-First Engineering:** Focus on clear domain models, SOLID principles, and clean class abstractions.
- Hybrid Evaluation Architecture:** Combine deterministic AST/regex structure analysis with an LLM qualitative evaluator behind an `IEvaluator` interface.
- Fail-Safe Resilience:** Provide deterministic fallback evaluation (`MockLLMEvaluator`) if external LLM APIs fail or time out, accompanied by one-click evaluation retries.
- Multi-Format Practice:** Allow learners to submit solutions in code (Java, TypeScript, Python), structured design text, or PlantUML diagrams.
- Zero-Friction Monolith:** Keep infrastructure minimal, simple, and self-contained so learners can run and practice anywhere without cloud setups.